

B.2 Implementation of Baselines

Goal Conditioning. We use the provided state-conditioning implementation in Robomimic [30] and condition the diffusion policy on an additional input consisting of the final observation in the trajectory. We train this policy using the same hyperparameters as the base diffusion policy.

Position Guidance (ITPS) [51]. The ITPS algorithm excels at steering the policy towards a target point in 3D specified by a human. To compare ITPS to our approach that does not require human specification, we manually compute the average final position of the robot for a set of 20 trajectories showing the desired behavior. We then use this position as the target point in ITPS.

Sampling (GBC) [40]. Previous work that introduced GBC-Rank used an objective function to rank action samples from a policy to select the best sample. We take that idea and apply it to our guidance metric (Eq. 2). We sample the base policy 5 times per inference and pick the best action to execute based on the metric.

Classifier-Free Guidance [20]. We use the same goal-conditioned architecture as the goal-conditioned baseline. However, during training, we randomly zero the goal condition to represent the unconditional input needed in classifier-free guidance. During inference, we compute the classifier-free guidance noise estimation using the same conditions used in the goal-conditioned policy baseline.

B.3 Distance Metric: Derivation and Design Choices

In this section, we will conduct a more detailed derivation of the metric shown in Eq. 2. While other works [57] simply assume the metric to be Euclidian, we use a probabilistic motivation to understand the log-sum-exp, which we discovered empirically to be very important for the performance of DynaGuide. For the first part of this derivation, we will only talk about the desired outcomes $\mathbf{z}^+$, and the undesired outcomes follow by negation.

We start from the very rough approximation that the latent space is Gaussian with diagonal variance Σ . It is an approximation that methods using squared latent distances already implicitly make. Here, the probability $p(z_i^+ | \hat{z}_{t+H}) = \mathcal{N}(z_i^+, \mu = \hat{z}_{t+H}, \Sigma = \sigma I)$. The log probability can be computed as follows:

$$\begin{aligned} \log p(z_i^+ | \hat{z}_{t+H}) &= \log \frac{1}{(2\pi)^{n/2} \sqrt{|\Sigma|}} \exp \left(-\frac{1}{2} (z_i^+ - \hat{z}_{t+H})^T \Sigma^{-1} (z_i^+ - \hat{z}_{t+H}) \right) \\ &= -\frac{1}{2} (z_i^+ - \hat{z}_{t+H})^T \Sigma^{-1} (z_i^+ - \hat{z}_{t+H}) + \underbrace{\log \frac{1}{(2\pi)^{n/2} \sqrt{|\Sigma|}}}_{\text{Constant}} \\ &= -\frac{1}{2\sigma} (z_i^+ - \hat{z}_{t+H})^T (z_i^+ - \hat{z}_{t+H}) + C \\ &= -\frac{1}{2\sigma} \|z_i^+ - \hat{z}_{t+H}\|_2^2 + C \end{aligned}$$

There are different ways of combining the influence of multiple z_i^+ in $\mathbf{z}^+$. One way that empirically *does not* work is adding the log-probabilities together:

$$d = \sum -\frac{1}{2\sigma} \|z_i^+ - \hat{z}_{t+H}\|_2^2$$

This is not a surprise, as adding log-probabilities is equivalent to *multiplying* the probabilities together. As previously discussed, not all the z_i^+ 's are achievable and some may be mutually exclusive. For example, in the multi-objective setup (§4.3), some z^+ might correspond to pressing the button and

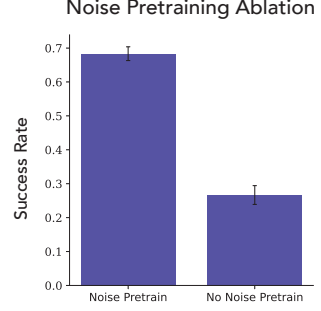


Figure 9: **Ablating Noise Pretraining.** Removing the dynamic model’s exposure to noised actions greatly decreases its ability to steer the action diffusion process.